

社内システム開発に伴うプログラミング言語選定 総合報告書

Windows隔離環境から将来のLinux移行まで見据えた「常時稼働・社内API連携バッチ」の総評

1. 確定した前提条件・システム要件

用途・形態

社内システム間のデータ連携(画面なしバッチ処理)

現在の稼働環境

Windows Server(本番は外部インターネット通信が不可能な隔離環境)

将来の拡張性

将来的にLinux(RHEL系等)サーバーへ移行する可能性あり

主要連携先

Oracle Database & 社内の別サービスAPI(イントラネット内通信)

起動頻度・運用

5分間隔などの高頻度で常時稼働する仕組み

最優先基準

現要員のスキルに縛られず、将来の「運用・長期保守性」を最優先する

2. 最終候補2案の多角的一覧比較

比較項目	【第一候補】C# (.NET 8 LTS) ／ バイナリ持参型	【第二候補】Go (Golang) ／ ソースコード持参型
常時稼働・5分間隔の実装形態	Windowsサービス(プロセス常駐) 標準の「Worker Service」機能でOS常駐アプリを即座に実装可能。メモリ上にDB接続やAPIトークンを保持(キャッシュ)できる。	タスクスケジューラ(都度起動) 5分ごとにOSが起動・終了させる運用に最適。「起動→送信→即終了」の使い捨て構造により、常駐によるメモリリークのリスクを排除。
将来のLinux移行 (ポータビリティ)	対応可能(ただし手直しあり) .NET 8はLinuxでも動作するが、「Windows サービス」として作った常駐部は、Linuxの`systemd` (デーモン)用にコードの書き直しが発生する。	最高評価(ほぼ修正不要) 元々Linux親和性が非常に高い。タスクスケジューラ運用にしておけば、Linuxの`cron`に実行ファイルを乗せ換えるだけでプログラム修正なく移行完了。
クロスコンパイル (隔離環境への展開)	Linux用バイナリの別ビルド 手元PCからLinux向けに Self-contained 形式で再ビルドして持ち込む。Windows用とは別アセンブリとして管理が必要。	圧倒的に容易 環境変数を1行変えるだけ(`GOOS=linux`)で、手元のWindowsからLinux用の単一実行ファイルを一瞬でクロスコンパイル可能。
Oracle接続方式 (Clientレス対)	ODP.NET Managed Driver 完全C#製の公式ドライバ。Oracle Client (C++製)をサーバーにインストールすること	Pure Go Driver (sijms/go-ora) Cコンパイラ(CGO)に依存しない純粋Go製ドライバ。Windows/Linuxどちらの環境でも

比較項目	【第一候補】C# (.NET 8 LTS) / バイナリ持参型	【第二候補】Go (Golang) / ソースコード持参型
応)	なく、`.exe` に完全内包可能。	そのまま動作する。
セキュリティ審査 (ファイル持ち込み)	審査が厳格な傾向 ブラックボックスなバイナリ(.exe)を持ち込むため、社内ルールによってはハッシュ値の事前登録や資産申請に時間を要する場合がある。	審査を通しやすい ソースコード(テキスト)と`vendor`内の外部コードをそのまま持ち込むため、テキストレベルでの静的解析や目視レビューが容易。
将来の保守性・要員	最高評価(Windows環境の標準) 国内の「Windows×Oracle」環境におけるエンジニア数が圧倒的。長期間(5～10年)のシステム維持・保守において要員確保に困らない。	コード平易性による属人化防止 言語仕様が小さく「誰が書いても同じコードになる」ため保守しやすい。ただし、社内業務バッチ分野での要員市場はC#より狭い。

3. 最終判断の総評とおすすめの選定方針

【結論】「将来的なLinux移行の現実度」と「社内のセキュリティルール」が最大の決定打になります。

現行のWindows Serverでの運用の安定性と、5～10年先の保守要員の捕まえやすさを最優先するならC#が手堅いです。しかし、数年以内にLinuxへの移行が具体化している、あるいは隔離環境への「バイナリ(.exe)持ち込み審査」が極めて厳しい社内風土である場合は、ポータビリティと透明性に勝るGoが逆転で最適解となります。

A案:C#を選択すべきシチュエーション

- Linux移行の可能性はあるがまだ不確定で、まずは現行のWindows Server環境で「最も社内実績が多く、トラブル時に社内ナレッジを頼れる体制」を重視する場合。
- 5分間隔の通信において、毎回APIの認証トークンを取得し直すのを避け、メモリ上にトークンやDB接続(コネクションプーリング)を常駐・キャッシュさせて効率化したい場合。

B案:Goを選択すべきシチュエーション

- Linuxへの移行時期がほぼ決まっている、または高確率で発生するため、OS移行時にプログラムの書き直しコストを「ゼロ」に抑えたい場合。

- Windowsのタスクスケジューラ(Linux移行後はcron)による「5分ごとの都度起動・都度終了(使い捨て)」の運用が社内ルール上、最も管理しやすい場合。

4. 次の議論(インフラ・API・セキュリティ担当者)への確認事項

1. 持ち込み規約の確認: 隔離環境のサーバーに対して、「中身の確認できないビルド済みの`.exe`」と、「テキスト状態のソースコード一式(Go現地ビルド案)」のどちらがセキュリティ審査をスムーズに通過できるか。
2. 常駐か都度起動かの運用ルール確認: 5分間隔のバッチを動かすにあたり、インフラ監視の観点から「Windowsサービスとして常駐」させるのと、「タスクスケジューラで5分おきに都度起動」させるの、どちらがサーバー運用ルール(エラー検知やリソース管理)に合致するか。
3. 通信経路の確認: 本番の隔離Windowsサーバーから、別サービスのAPIサーバー(社内IP)へのポート疎通(一般的にはポート80または443)が現状の社内ネットワークで許可されているか。